NPN Age Verification - Team Guide

Internal reference for the 8-person hackathon team. Written so a teammate who has never trained a model can read it and understand what we built, why, and how to run it.

Project: age estimation from facial images, built for the Cognizant NPN hackathon (healthcare domain, "Age Prediction" track). Repo: `C:/Users/prana/Projekts/NPN`.

---

1. What this project is

The one-line pitch

Upload a face photo. The system estimates an age, decides whether that age clears a policy rule (for example, "18 or older"), and - when it is not confident enough to decide alone - sends the case to a human reviewer instead of guessing. Every decision is logged, but the photo itself is never saved.

Why a bare age number is not enough

An age regressor (a model that predicts a number) just outputs a number, like "34.2". A clinical workflow cannot act on a bare number - it needs to know whether to trust it. A model that is 4 years off on average is 4 years off in *both* directions, and near a policy boundary (say, trial eligibility at 18) that error can flip the decision from "eligible" to "not eligible" or back. Reporting one accuracy number and stopping there hides exactly the cases that matter most: the ones near a boundary, or the ones the model is quietly unsure about.

So the system does three things a plain model does not:

1. **Bands the estimate into a clinical decision** - eligible, ineligible, or undecided.
2. **Routes uncertain cases to a human** instead of auto-deciding. This fires when the model's confidence is unusually low, or when its predicted age range straddles a band boundary (a confident estimate that still sits right on a boundary is not decisive either).
3. **Logs every decision with a fingerprint of the image, never the image itself.**

The model (the ML part) is deliberately the small half of this project. The deliverable that matters is the decision path wrapped around it: age estimate to clinical band to policy decision to human review routing to audit trail. Judge changes to this project by whether that path stays honest, not by whether the accuracy number moved.

Two things to protect

The **repo** and the **demo recording**. Both outlive the hackathon as portfolio artifacts - that matters more than the hackathon prize itself.

---

2. How to run it

Two commands after setup. One process serves both the API and the built frontend on one port - nothing else needs to run.

Windows paths use `.venv/Scripts/`; on Linux/macOS use `.venv/bin/` instead.

```
# 1. set up the Python environment
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements.txt

# 2. build and install the frontend
cd web && npm install && npm run build && cd ..

# 3. run the one server (serves API + built UI on port 8000)
.venv/Scripts/python.exe -m uvicorn server.main:app --port 8000
```

Open `http://127.0.0.1:8000`. By default it runs in **mock mode** (`NPN_MOCK=1`) - a synthetic, no-model version that returns realistic-shaped fake answers, so the frontend team never has to wait on the model team. A `SYNTHETIC MODEL - NOT FOR CLINICAL USE` badge shows on screen whenever mock mode is active.

Frontend dev loop (hot reload while iterating on the UI):

```
cd web && npm run dev
```

This proxies `/api` calls to port 8000, so the uvicorn server above must already be running.

Checks

There is no pytest/vitest test suite - each module carries its own small `assert`-based self-check you run directly:

```
.venv/Scripts/python.exe server/bands.py       # decision + routing logic
.venv/Scripts/python.exe server/store.py       # audit/queue + no-image-retention
.venv/Scripts/python.exe ml/gate0.py --selfcheck
.venv/Scripts/python.exe tests/test_api.py     # 7 contract tests
cd web && npx tsc -b                           # TypeScript types only
cd web && node scripts/shots.mjs               # captures docs/shots/*.png (server must be
up)
```

To run just one of the 7 API contract tests, pass a substring of its name:

```
.venv/Scripts/python.exe tests/test_api.py queue      # runs test_review_queue_roundtrip
.venv/Scripts/python.exe tests/test_api.py envelope
```

Running the trained model instead of mock mode

Everything above runs the server in mock mode (`NPN_MOCK=1`), which needs no `torch` install at all. To serve real predictions from the checkpoint already committed to the repo (`checkpoints/dist-v1`), install the ML dependencies separately - they're kept out of the default `requirements.txt` install specifically so the API can boot on a machine with no GPU or ML stack:

```
# CPU-only machines: plain pip install works
.venv/Scripts/python.exe -m pip install torch torchvision
.venv/Scripts/python.exe -m pip install timm==1.0.12 opencv-python==4.10.0.84

# then run with mock mode turned off
NPN_MOCK=0 .venv/Scripts/python.exe -m uvicorn server.main:app --port 8000
```

RTX 50-series GPUs (Blackwell, `sm_120`) need a different torch build. A plain `pip install torch` pulls the cu124 build and fails at runtime with "no kernel image is available for execution on the device" - it installs fine and only breaks the moment you try to use the GPU. Use the cu128 wheels instead:

```
.venv/Scripts/python.exe -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu128
```

`server/main.py::_load_predictor` only imports `ml.predict.Predictor` the first time a real prediction is requested, so none of this is needed just to run the API in mock mode - only to serve the actual trained model.

Both the mock-mode and real-model contract test suites were run and verified while writing this guide: all 7 tests pass with `NPN_MOCK=1` (the default) and all 7 pass again with `NPN_MOCK=0` against the committed `checkpoints/dist-v1` weights.

---

3. Architecture overview

Three pieces, glued by one frozen contract:

- **The API contract** (`contract/predict.contract.md`) - the agreed shape of every request and response, frozen before either side started building.
- **The backend** (`server/`) - FastAPI. Owns the decision logic and the audit database, and serves the built frontend files too.

- **The frontend** (`web/`) - React. Talks only to the backend's `/api/*` endpoints, never assumes anything the contract doesn't say.
- **The ML lane** (`ml/`) - trains the model and, at serving time, plugs into the backend behind a lazy import (`server/main.py::_load_predictor`) so the whole API can boot and run in mock mode even with no `torch` installed.

Request flow for a real prediction:

```
                upload photo
                     |
                     v
      FastAPI  /api/predict   (server/main.py)
                     |
      hash image -> SHA-256 digest, then DELETE the bytes
                     |
                     v
     ml/predict.py :: Predictor.predict(bytes)
      - find the face (or fall back to full frame)
      - run EfficientNet-B0 -> distribution over age bins
      - decode distribution -> age, interval, confidence
                     |
                     v
      server/bands.py :: decide()
      - band the age (paediatric / young_adult / adult / older_adult / geriatric)
      - apply policy (e.g. trial_eligibility_v1: 18-64)
      - route to review if confidence is in bottom 15%,
        OR the interval straddles a band boundary
                     |
                     v
      server/main.py :: _envelope()   <- single place the response is built
                     |
        +------------+------------+
        |                         |
  store.py: write audit    return JSON envelope
  row (digest only,        to the browser
  never image bytes)              |
                                  v
                        React UI renders result,
                        band ladder, review flag
```

In **mock mode**, the "run the model" step is replaced by a deterministic fake based on the image's digest - same photo always gives the same fake answer, which is what makes demo runs reproducible even before the model exists.

---

4. The frozen contract

`contract/predict.contract.md` (version `1.0.0`) is the single source of truth for what the API looks like. It was **frozen at Day 1, Hour 1** - before either the frontend or the ML lane had written real code - specifically so the frontend team could build the whole UI against fake/mock responses while the model was still training. Nobody had to wait on anybody else.

Three files must always change together, or the freeze is meaningless:

- `contract/predict.contract.md` - the prose description (the actual source of truth)
- `web/src/api.ts` - the TypeScript types that mirror it exactly
- `tests/test_api.py` - tests that check the running server against the contract document, not against whatever the code happens to currently do

The response envelope

Every single status returns the exact same JSON shape - the fields are just filled in differently (many of them `null`) for the failure cases. This matters: the frontend only ever needs to handle one shape, never a special case per failure type.

```
{
  "request_id": "3f9a1c2e-...",
  "status": "ok",
  "age_estimate": 34.2,
  "age_interval": [29.1, 39.3],
  "confidence": 0.781,
  "confidence_percentile": 0.42,
  "band": { "id": "adult", "label": "Adult (30-49)", "min": 30, "max": 49 },
  "decision": { "outcome": "verified", "reason": "...", "policy": "trial_eligibility_v1" },
  "review_required": false,
  "face_box": [64, 48, 192, 192],
  "model": { "name": "efficientnet_b0", "version": "1.0.0", "head": "dist_bins" },
  "latency_ms": 41
}
```

The five statuses

STATUS	MEANING	UI SHOWS
`ok`	a face was found and a prediction was made	the result screen
`no_face`	no face detected in the image	"No face detected" + retry
`multi_face`	more than one face found	"Multiple faces - submit a single subject"
`low_quality`	a face was found but it's too small or blurry	"Image quality insufficient" + reason
`error`	something failed server-side	error state, with `request_id` for the audit trail

Whenever `status` is not `"ok"`, the numeric fields (`age_estimate`, `age_interval`, `confidence`, `confidence_percentile`, `band`) are all `null`, `decision.outcome` becomes `"indeterminate"`, and `review_required` is forced to `true`. In plain words: **an image the system can't read never gets auto-decided - it always goes to a human.**

`no_face`, `multi_face`, `low_quality`, and `error` are treated as normal, expected outcomes with their own designed screens - not as exceptions or crashes.

---

5. Backend

`server/bands.py` - bands, policy, and the review-routing rule

This one file is the single source of truth for age bands and clinical decisions. Nothing else in the codebase is allowed to hardcode an age boundary.

Active bands (`ACTIVE_BAND_SET = "lifespan"`):

BAND	AGE RANGE
paediatric	0-17
young_adult	18-29
adult	30-49
older_adult	50-64
geriatric	65+

`band_for(age)` always returns a band - even an out-of-range age (say, -3 or 150) gets clamped to the nearest edge band rather than returning nothing. The reasoning: an unbanded prediction has no decision path, so it must always land somewhere.

Review routing rule: `REVIEW_PERCENTILE = 0.15`. A prediction is routed to manual review when *either*:

1. its confidence sits in the **bottom 15%** of the confidence scores measured on the validation set, or
2. its predicted age interval **straddles a band boundary** (e.g. the interval [16, 19] straddles the 18-year-old paediatric/young_adult line).

Why a **percentile** and not a raw confidence number (like "route anything below 0.30")? Because a raw cutoff has to be re-picked by hand every time the model changes and the confidence scale shifts. A percentile ("bottom 15% of what we've actually seen on real validation data") stays defensible no matter what the raw confidence numbers happen to look like on a small validation set.

The one hard rule in this file: *review beats verified and rejected.* Even if an age looks perfectly inside policy range, if the confidence is low or the interval straddles a boundary, the case is routed to review - never auto-decided. This is the entire point of the human-in-the-loop design: an uncertain prediction must never be auto-actioned.

`server/store.py` - audit log and the no-image-retention rule

SQLite database with two tables: `audit` (every event that happens) and `review_queue` (cases waiting for a human).

The hard rule, enforced in code, not just claimed on a slide: image bytes are never persisted. Here's the mechanism in plain terms: when a photo is uploaded, the server immediately runs it through a one-way "fingerprint" function called **SHA-256** - think of it as a scrambler that turns any file into a fixed 64-character string of letters and numbers. The scrambled string always comes out identical for the identical file, but you can't reverse it back into the photo. That fingerprint (called a "digest") is what gets stored in the database - never the actual image bytes.

`server/store.py::hash_image()` is the *only* place in the whole codebase that touches raw image bytes. Every other function in the store module takes a digest **string** as input - there's no code path that could even accept raw bytes if it wanted to. `server/main.py::predict()` computes the digest and then explicitly deletes the image data (`del data`) right after, before anything else runs.

This is the answer to "is this HIPAA-safe?" - and `tests/test_api.py` has an automated test (`test_audit_stores_digest_never_bytes`) that opens the actual database file after a prediction and asserts the raw bytes are nowhere in it.

One more small but important gotcha fixed here: `store._conn()` is written as a "commit-and-close" wrapper. A plain `sqlite3.connect(...)` used directly as a Python context manager commits the transaction on exit but does **not** close the connection

- which leaks a database handle every single call, and on Windows that eventually locks the whole database file. Don't revert this pattern if you touch `store.py`.

`server/main.py` - the single envelope construction point, and mock mode

`server/main.py::_envelope()` is the **one and only** place in the entire codebase that builds the response JSON described in the contract. If a new field needs to be added to every response, it goes here - never inside an individual route handler. This is what keeps every status (`ok`, `no_face`, etc.) guaranteed to return the exact same shape.

Mock mode (`NPN_MOCK=1`, the default) returns realistic contract-shaped answers without needing a trained model at all, so the frontend was never blocked waiting on training. Mock answers are deterministic based on the image's SHA-256 digest - the same photo always produces the same fake answer, every single run, which is also what makes demo rehearsals reproducible.

Two digest prefixes are reserved as fixtures so every UI state can be exercised without a real model: any image whose digest happens to start with the character `0` returns `no_face`, and any starting with `1` returns `low_quality`. Everything else gets a plausible fake age.

The model itself is loaded lazily (`server/main.py::_load_predictor`) - the import of `ml.predict.Predictor` only happens the first time a real (non-mock) prediction is requested, so the whole API server can start up and run fine even on a machine with no `torch` installed.

---

6. Frontend

Four views, no router

`web/src/App.tsx` uses plain `useState` to switch between four views (no router, no state management library - the app is intentionally small). Note: `PRODUCT.md` and `CLAUDE.md` describe "three views" - that was true at the start of the build; the fourth view (Model evidence) was added afterward once the calibration and risk-coverage work landed, and those docs haven't been updated to match. The actual running app has four:

1. **Verify** - upload a photo, see the assessment
2. **Review queue** - cases the model declined to decide alone
3. **Audit trail** - every prediction and adjudication ever logged (digest only)
4. **Model evidence** - the accuracy and calibration numbers behind the model, so a panel member can check the claims rather than trust them

Design system: "clinical instrument panel," not a web app

The design brief was explicitly *"professional interface, not AI slop."* Key rules (documented fully in `DESIGN.md`):

- **Light clinical paper**, never a dark console. A warm off-white background, white panels separated by thin hairline rules instead of cards or shadows.

- **Banned outright:** pure black, any purple or violet, gradients, glow, glassmorphism. These are the visual tells that make an interface look AI-generated - avoiding them is a hard rule, not a preference.
- **One accent color** - an ochre/amber tone (`--color-signal`, `#b06a12`). It means exactly one thing: "a human must look at this." It is never used decoratively. Outcome colors (verified/rejected/etc.) are treated as data, not decoration.
- Every number on screen renders in **IBM Plex Mono** with tabular numerals, so digits line up in columns like a printed lab report. Labels and prose use Instrument Sans.
- Fonts are self-hosted (not loaded from a CDN), because the demo laptop runs fully offline and a CDN font link would silently fall back to an ugly serif font in front of the panel.

`web/src/BandLadder.tsx` - the differentiating visual

This is the one piece of custom visualization in the whole app, and it earns its place. It draws the clinical age-band scale as a horizontal bar, overlays the model's predicted age *interval* on top of it, and marks the single point estimate. If the interval crosses a band boundary, that boundary line lights up in the ochre accent color.

The point: it *shows* why a case was routed to human review, instead of just printing "routed to review" as text. A panel member can look at the picture and immediately see "the interval spans the 18-year-old line, that's why this one went to a human" without reading any prose.

`web/src/api.ts` mirrors the contract exactly

Every TypeScript type in this file (`Status`, `Outcome`, `Prediction`, `Meta`, etc.) is a direct mirror of `contract/predict.contract.md`. The file's own comment states the rule plainly: *"These types ARE the contract. If the server drifts, this file must change in the same commit."* The frontend never hardcodes a metric number either - it always reads `/api/meta` for bands, policy, and accuracy numbers, so nothing on screen can silently go stale relative to the actual model.

---

7. Machine learning, explained simply

The dataset

233,200 images total, ages 1 to 100, sourced from a public Kaggle dataset (`mariafrenti/age-prediction`). Early on there was real confusion: Kaggle's page said ages 1-100, but a public code example online only used a 20-50 subset. Both turned out to be true - **the dataset actually ships two separate, independent sets of folders**: one covering the full 1-100 lifespan, and a smaller one covering just ages 20-50. We use the full lifespan set.

This got checked *before* deciding how to band ages (called "Gate 0" in the code, `ml/gate0.py`) - rather than assuming a range and finding out we were wrong on stage. The measured counts (`ml/dataset_census.json`) show the training split has 185,632 images, ages 1-100, with under-18 subjects at 4.4% of the data and over-64 subjects at 4.9%. Both were judged large enough (over a 2% floor) to support building clinical bands across the whole lifespan rather than just adults. One caveat worth remembering: the very oldest ages (90+) are only 273 images in the training data - thin enough that we always report accuracy per band rather than one single overall number, so that thin tail doesn't get hidden inside a good-looking average.

The model: EfficientNet-B0

EfficientNet-B0 is a well-known, relatively small and fast image classification network (originally built to recognize objects in photos, like "cat" or "car"). Rather than designing a brand-new network from scratch, we start from a version that's already been trained on millions of general photos ("ImageNet-pretrained") and retrain it on faces and ages - a very common and efficient technique called **transfer learning**. It already knows how to see edges, textures, and shapes; we're teaching it to turn "a face" into "an age" instead of "a cat."

Distribution over age bins, instead of one number

The obvious approach would be: train the model to output a single number (an age), and compare it to the true age. This is called **scalar regression**, and the code has it too as an honest baseline. But it has a real weakness: a model trained this way tends to "hedge" toward the average age it's seen, especially at the extremes (very young or very old), and it gives you no natural sense of *how sure* it is.

Instead, our default approach has the model output **a probability for every single year of age from 1 to 100** (technically, "one softmax bin per year") - like a little bar chart showing how likely the model thinks the subject is 30, 31, 32, and so on. The final age estimate is just the **expected value** of that bar chart (its weighted average). But because the model produces a whole distribution instead of one number, we get three things for free:

- **the point estimate** - the distribution's average
- **an 80% interval** - the range that captures the middle 80% of the model's probability mass (its 10th to 90th percentile), i.e. "the model thinks the true age is probably somewhere between 29 and 39"
- **a confidence score** - how "peaked" (concentrated) versus "flat" (spread out) that bar chart is

That confidence and interval are exactly what the review queue needs to decide whether to trust a prediction or send it to a human.

Soft labels

During training, the model isn't just told "this photo is exactly age 35" as a hard fact. Instead it's trained against a **soft label** - a small bell curve (a Gaussian) centered on 35, so the model is also told "36 and 34 are nearly right too, just a little less right." This matters because a plain classifier (told only "this is exactly 35, nothing else") would treat guessing 34 for a 35-year-old as just as wrong as guessing 90 - which throws away useful information about how close a guess is.

We independently arrived at a published method: DLDL

After building this, a piece of research (documented in `docs/RESEARCH.md`) found that what we'd built already has a name in the academic literature: **Deep Label Distribution Learning (DLDL / DLDL-v2)**, a well-regarded, published technique (IJCAI 2018, building on earlier work called DEX from 2015). The literature names DLDL-style methods as one of the two families of techniques that beat both plain regression and plain classification for age estimation - for exactly the reasons described above: plain regression collapses toward the average, and plain classification can't express "34 is nearly 35."

We built this before finding the paper, which is worth saying plainly in the deck: it's a validated, citable method, not an improvised one-off.

---

8. The metrics - what each number means

Plain-English definitions first

- **MAE (Mean Absolute Error)** - on average, how many years off the model's guess is from the true age, ignoring whether it guessed too high or too low. Lower is better. An MAE of 5.6 means: on average, across every test photo, the guess was off by about 5.6 years.
- **CS@5 (Cumulative Score at 5 years)** - the percentage of predictions that were within 5 years of the true age. Higher is better. This is a common way of reporting "close enough" accuracy in the age-estimation research field.
- **Band accuracy** - the percentage of predictions that landed in the *correct clinical band* (paediatric / young_adult / adult / older_adult / geriatric), even if the exact number was slightly off. This is the number that matters most for the actual product, since the clinical decision only cares which band you're in.
- **Baseline** - what you'd get if the model just always guessed the *average* age of everyone in the training set, no matter what photo it saw. Any real model has to beat this by a wide margin, or it's not actually learning anything from the image.

Our actual results (test set, n = 47,568 held-out images the model never trained on)

METRIC	RESULT	BASELINE
MAE	**5.639 years**	11.336 years
CS@5	**59.3%**	-
Band accuracy	**66.8%**	-

In plain terms: our model is off by about 5.6 years on average, which is roughly half the error of just guessing the average age for everyone (11.3 years). It's within 5 years of the true age about 6 times out of 10, and it puts people in the correct clinical age band about two-thirds of the time.

Per-band MAE - why this matters more than the single headline number

BAND	N (TEST IMAGES)	MAE (YEARS)
paediatric	2,937	6.20
young_adult	13,121	4.98
adult	23,301	4.80
older_adult	5,933	7.60
geriatric	2,276	12.25
geriatric_90plus (subset)	77	15.95

A single "MAE: 5.6 years" headline hides that the model is actually **more than twice as accurate on adults (4.80 years) as on the geriatric band (12.25 years)**, and worse still on the tiny 90+ subgroup (15.95 years, only 77 images). Reporting per-band numbers instead of hiding them behind one average is a direct, deliberate honesty choice - a panel member asking "how good is this for elderly patients specifically?" gets a real answer instead of a dodge.

---

9. Calibration and the review queue

This section answers a harder, more important question than plain accuracy: **can the model's own stated confidence be trusted?** If low-confidence predictions aren't actually the wrong ones, then the whole review-queue idea is just theater - a reassuring-looking feature that doesn't actually do anything.

The pre-registered threshold

Before training even finished, the team wrote down in code *in advance* what would count as proof the confidence score is meaningful (this is called "pre-registering" - deciding the bar for success before you see the result, so you can't quietly move the goalposts afterward):

- the model's least-confident 10% of predictions (bottom decile) must have an MAE at least **1.30x worse** than its most-confident 10% (top decile)
- the relationship between confidence and error must be **monotonic** (error should consistently fall as confidence rises, not bounce around) in at least **70%** of the ten confidence buckets ("deciles")

Result: it passed, and passed comfortably. The actual bottom-to-top ratio came out at **3.888x** (nearly 3x better than the 1.30x bar), and the relationship was **perfectly monotonic (100% of deciles)** - error fell at every single step from least confident to most confident.

The decile table

Ten equal-sized buckets, sorted from least confident (decile 1) to most confident (decile 10):

DECILE	1	2	3	4	5	6	7	8	9	10
MAE (yr)	10.12	7.85	6.76	6.18	5.47	5.12	4.60	4.02	3.66	2.60

In plain words: the 10% of predictions the model is *least* sure about are, on average, off by about 10 years - while the 10% it's *most* sure about are off by only about 2.6 years. The confidence score is doing real, measurable work: it correctly tells you which predictions to trust more.

Calibration curve - does an "80% interval" actually mean 80%?

The decile table above proves the confidence score *ranks* predictions correctly (more confident = less wrong). It does **not** prove something subtly different: whether a stated "80% interval" (the range shown on screen) actually contains the true age 80% of the time. That's a different, stronger claim called **calibration**, and it was checked separately:

NOMINAL (CLAIMED)	EMPIRICAL (ACTUAL)
50%	52.4%
60%	61.3%
70%	70.3%
80%	79.1%
90%	88.2%
95%	93.1%

Maximum deviation between claimed and actual: **2.4 percentage points**. In plain words: when the system says "I'm 80% sure the true age is in this range," the true age really does fall in that range about 79-80% of the time. The uncertainty shown on screen is honest - it's not just decoration, it means what it says.

Risk-coverage - does deferring the hardest cases actually help?

This answers: if we send the least-confident cases to a human instead of letting the model decide, does accuracy on the *remaining, auto-decided* cases actually improve? ("Coverage" = the fraction of cases the model still decides on its own after the worst cases are deferred to review.)

- **Full coverage (deciding on 100% of cases) MAE: 5.639 years** (the same headline number as before)
- **MAE at 85% coverage (deferring the worst 15% to review): 4.991 years**
- That's an **11.5% improvement** in accuracy on the cases the system still handles itself, just by routing away the 15% it was least sure about.
- **AURC** (Area Under the Risk-Coverage curve - a single number summarizing the whole curve, lower is better) is **3.945**, compared to a theoretical **perfect oracle** (an idealized system that always defers exactly the actually-wrong predictions) at **2.122**.

In plain words: the review queue is doing real, measurable work, not just adding an extra step for show. Deferring the 15% of cases the model is least sure about makes the remaining 85% meaningfully more accurate.

These numbers matter more than the plain MAE, because the entire product is built around the claim "the system knows when to ask for help." These are the numbers that prove that claim rather than assert it.

---

10. Honest limits

Stated here rather than left for a reviewer or panel member to discover:

- **Image-only.** The original hackathon problem statement also mentioned text and voice as possible signals. No usable dataset was available for those, so a second modality was deliberately scoped out rather than faked.
- **No demographic fairness numbers.** The reference dataset carries no skin-tone or ethnicity labels, so a per-demographic-group fairness breakdown cannot be computed from it at all. This is a real, acknowledged gap for a biometric healthcare system - closing it would require an external, labeled, balanced evaluation dataset (e.g. FairFace). Instead, results are broken down by age band, which the data does support.
- **Geriatric weakness.** The geriatric band (12.25 years MAE) is roughly 2.5x worse than the adult band (4.80 years), and the 90+ subgroup (15.95 years, only 77 test images) is thinner still. This is reported plainly rather than hidden in an average.
- **Not for clinical use.** This is a research/hackathon demo. Age estimation from faces carries real bias and consent concerns and is not a substitute for a documented date of birth in any real clinical setting.

- **The train/serve framing gap.** The face-detection pipeline (`ml/predict.py`) uses an old, lightweight, offline-capable technique called a **Haar cascade** (bundled with OpenCV, no download needed). Measured on 300 training images, it only actually found a face in **231 of 300 (77%)** of clean, already-cropped photos - the other 23% it missed entirely. Since a detector that misses nearly 1 in 4 real faces can't be trusted as a hard gatekeeper, the system never treats "no face found" as proof there is no face: it predicts on the full frame anyway, and **forces that case into the review queue** rather than either confidently guessing or flatly rejecting it. The "review beats verified and rejected" rule from section 5 applies here too.

---

11. Where things stand and what is left

Done:

- Trained model committed to the repo (EfficientNet-B0, distribution-over-bins head)
- Test MAE 5.639 years on 47,568 held-out images (see section 8)
- Pre-registered review-queue calibration threshold cleared (3.888x vs a 1.30x bar, 100% monotonic vs a 70% bar) - see section 9
- Calibration curve and risk-coverage evidence measured and shipped in the Model Evidence view
- 7 out of 7 contract tests pass, in both mock mode and real (trained-model) mode
- Research report confirming the architecture matches a published method (DLDL) and naming the review-queue framework (selective prediction)

Remaining work (per `PLAN.md`, lanes 7 and 8):

- The **backup demo recording** (a pre-recorded video of the full happy-path demo, in case Wi-Fi, the laptop, or the API fails live on stage)
- The **deck** (slides, architecture diagram, scope-decision slide)

---

12. Glossary

Plain-English definitions, in the order they matter to this project:

- **MAE (Mean Absolute Error)** - the average size of the model's mistake, in years, ignoring the direction (too old vs too young) of the mistake. Lower is better.
- **CS@5** - the percentage of predictions that landed within 5 years of the true age.
- **MAE baseline** - the error you'd get from a "dumb" model that always guesses the average age from the training data, no matter the photo. The real model must beat this comfortably to

prove it's actually learning from the image.

- **Epoch** - one full pass of the model through the entire training dataset. Training runs for several epochs, and accuracy is checked after each one.
- **Held-out / test split** - a chunk of the data that is set aside and never shown to the model during training, used only to measure how well it generalizes to photos it's never seen.
- **Validation split** - a second, smaller held-out chunk (carved out of the training data, separate from the test split) used *during* training to decide things like "which epoch's version of the model was actually best" and to measure the confidence distribution used for review routing.
- **Overfitting** - when a model gets very good at the training data but doesn't actually generalize - it memorized instead of learned. Checking accuracy on a held-out split is how you catch this.
- **Backbone** - the main body of the neural network that does the heavy lifting of "looking at" the image (here, EfficientNet-B0). A small custom "head" is attached on top of it to turn its output into an age prediction.
- **Transfer learning** - starting from a model already trained on a large, different dataset (here, general photos via ImageNet) and retraining it for a new, narrower task (here, faces to ages), instead of training a network from nothing.
- **Soft labels** - training targets that aren't just "the one right answer" but a small bell curve around it, so the model is also told that a nearby guess is "almost right" rather than "completely wrong."
- **Calibration** - whether a model's *stated* confidence (e.g. "I'm 80% sure") matches its *actual* correctness rate (does the true answer really fall in that range 80% of the time?). A model can rank its own mistakes correctly (see "decile table," section 9. without being calibrated - they are two different claims.
- **Coverage** - in a system that can defer/abstain on hard cases, the fraction of total cases it still decides on its own (as opposed to routing to a human).
- **Percentile** - where a value ranks relative to a whole distribution of measured values, expressed as "bottom X%" or "top X%," rather than a fixed raw cutoff number. Used here for the review-routing threshold so the rule doesn't need re-tuning every time the model changes.
- **Selective prediction** - the academic/research term for a system that can decline to answer ("abstain") on cases it isn't confident about, instead of always giving a best guess. Our review queue is an implementation of this idea.
- **TTA (Test-Time Augmentation)** - running a prediction multiple times on slightly altered versions of the same image (e.g. flipped horizontally) and combining the results, to squeeze out a small accuracy improvement without retraining anything.
- **Checkpoint** - a saved snapshot of a trained model's weights, so it can be loaded and used later without retraining.
- **Inference** - running an already-trained model on a new input to get a prediction (as opposed to training, which is the process of teaching it in the first place).

- **Contract envelope** - the fixed JSON response shape every API call returns, regardless of whether the outcome was a success or one of the defined failure states. See section 4.